

UrbanCart Analytics Pipeline

From Raw, Imperfect Data to Decision-Ready Business Insight

Prepared for UrbanCart Leadership

Big Data Term Project

Submission deadline: August 2, 2026

Data sources: ecommerce.db, legacy_customers_export.csv, product_catalog_2024.csv

Table of Contents

Executive Summary.....	3
1. Introduction and Business Context.....	5
2. Data Description.....	7
3. SQL Extraction and Methodology.....	10
4. Data Cleaning and Integration.....	20
5. NumPy Analytical Methods.....	28
6. Findings and Business Insights.....	36
7. Limitations and Threats to Validity.....	44
8. Recommendations and Next Steps.....	46
Appendices.....	48

Executive Summary

UrbanCart asked for an end-to-end analysis of customer value, product profitability, demand risk, and the trustworthiness of customer and product master data. The pipeline joins six SQLite tables with two deliberately messy CSV extracts, preserves audit flags for every material cleaning decision, and produces reusable processed files, numerical models, and management-ready charts.

- The Loyal segment generated the most cleaned completed-order revenue at \$2,334,550.52. This segment combines recent purchasing, high order frequency, and high monetary value.
- After cleaning three extreme price records, the leading revenue category changed from Books & Media in raw data to Apparel in cleaned data. This is the most consequential data-quality finding.
- Electronics achieved the highest effective margin at 52.40% after discounts, negative-quantity returns, and product cost.
- The relationship between average product rating and repeat-purchase rate was 0.076, indicating that rating alone is not a strong predictor of repeat behavior.
- The highest observed seven-day session-to-purchase conversion was 2.85% for mobile / France. The two-month linear trend forecast is \$234,675.01 for 2025-01 and \$239,406.69 for 2025-02, but the low R-squared of 0.893 requires cautious use.

Priority actions are to correct source price controls, protect high-value customers with segment-specific retention, use inventory reorder points for the highest-risk SKUs, and treat the regression as a baseline rather than a production forecasting model.

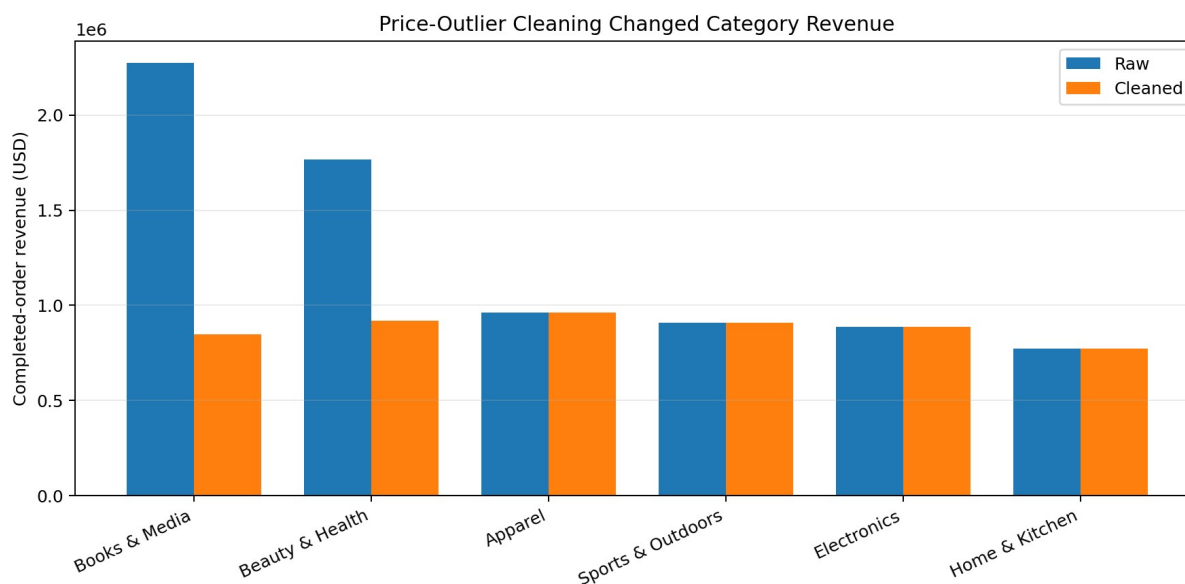


Figure ES-1. Raw versus cleaned completed-order revenue by category.

Executive Summary - Recommended Management Actions

The analysis supports five immediate decisions. First, create a data-entry validation rule that rejects product list prices beyond a reasonable relationship to cost or peer prices. Second, use the RFM segment file as a campaign audience table, beginning with retention and appreciation offers for Champions and Loyal customers. Third, replenish the five products with the highest simulated stockout probability using the recommended reorder-point file. Fourth, compare device/country conversion every month using the same seven-day attribution rule. Fifth, replace the simple linear forecast with a seasonal model when more history becomes available.

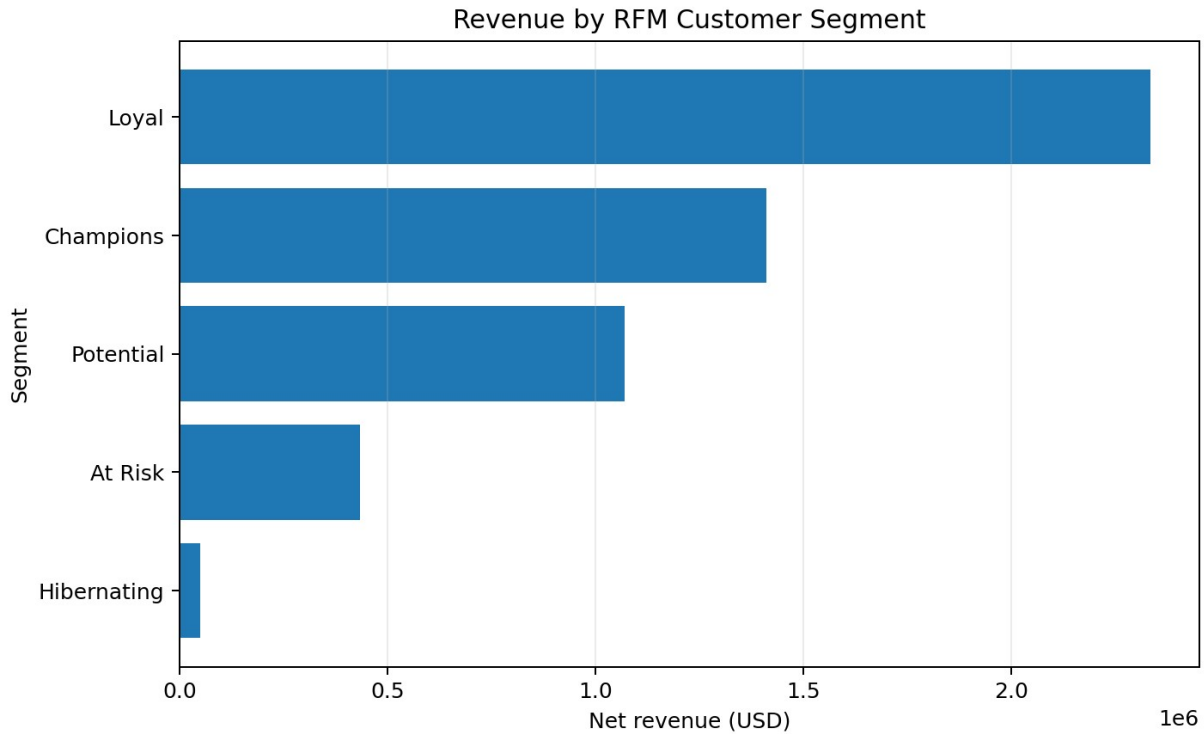


Figure ES-2. Revenue contribution by RFM segment.

1. Introduction and Business Context

1.1 Business Questions

UrbanCart is a fictional mid-size online retailer operating across fifteen cities and six categories. Leadership needs answers that are actionable rather than merely descriptive. The first question is which customers create the most value and what distinguishes customers who remain active from those at risk of churn. The second question is which products and categories create genuine profit after discounting, returns, and product cost, and whether short-term demand can be estimated. The third question is whether marketing and signup information from the old CRM and current database is reliable enough to support decisions.

These questions matter because revenue growth can be misleading when returns, duplicate rows, extreme price values, and customer duplication are ignored. A customer strategy also fails when identity resolution is weak. This project therefore treats data cleaning as part of the analytical result, not as invisible preparation.

- Customer value: recency, frequency, monetary value, demographic profile, and repeat behavior.
- Product economics: adjusted revenue, cost, margin, return rate, and stockout exposure.
- Data trust: duplicates, missing values, invalid ratings, inconsistent dates, product-source mismatches, and price anomalies.

1.2 Analytical Approach

The project follows a reproducible sequence. SQL extracts directly from ecommerce.db answer ten defined management questions. pandas then cleans and integrates the database tables with the legacy customer and supplier catalog files. NumPy implements quintile scoring, cosine similarity, the normal equation, and Monte Carlo simulation without shortcut modeling libraries. Finally, the cleaned data and numerical outputs are translated into eight charts and business recommendations.

Recognized revenue uses completed orders. Negative quantities remain negative and therefore reverse revenue and cost as return adjustments. Cancelled and pending orders are excluded from booked revenue. Historical line prices for three products identified as extreme IQR outliers are replaced for analysis with the non-outlier median of the same subcategory, while raw values and flags are retained for audit.

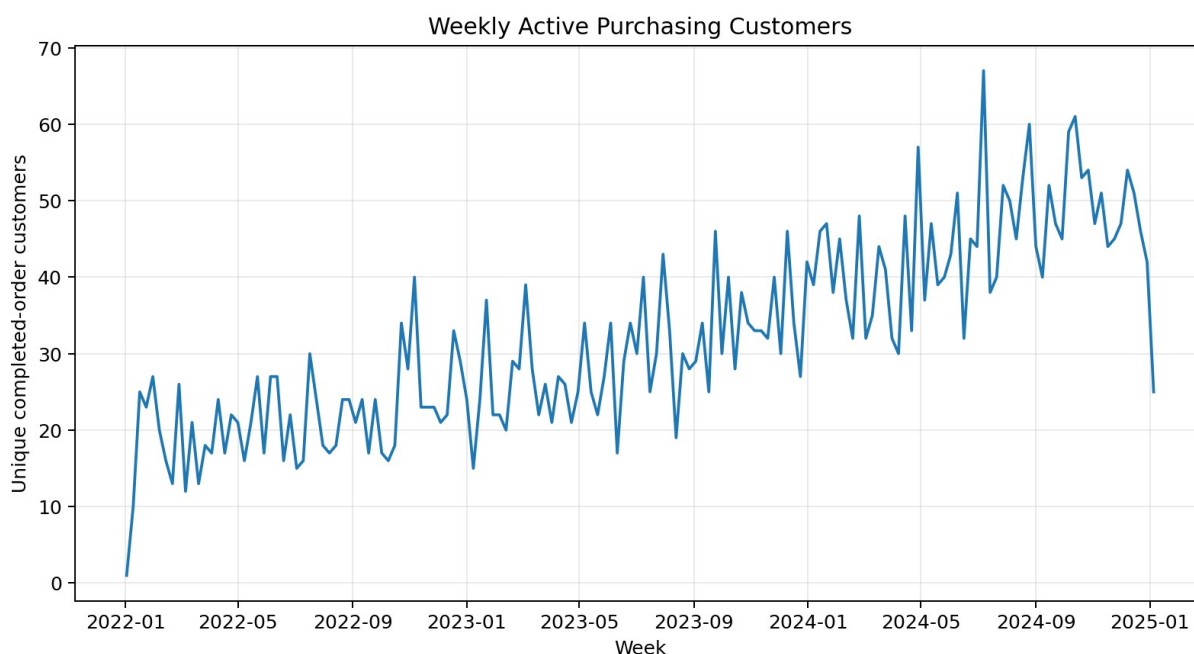


Figure 1. Weekly active purchasing customers over the available order history.

2. Data Description

2.1 Sources and Scale

The SQLite database contains six linked tables. The two CSV files are external sources and are not assumed to be clean or fully aligned with the database. The table below summarizes the record counts loaded by the pipeline.

Source	Rows
customers	2500
products	300
orders	9000
order_items	20362
reviews	4000
web_sessions	12000

2.2 Relational Schema and Analytical Grain

customers is the customer master and joins to orders, reviews, and web_sessions by customer_id. orders is the order header and joins to order_items by order_id. products joins to order_items and reviews by product_id. The product catalog CSV uses SKU as the equivalent of product_id. The legacy customer export has no stable numeric key, so normalized email is the primary match key and normalized name plus city is the secondary key.

The principal financial grain is one cleaned order-item row. Customer value is aggregated to one row per customer. Product recommendation uses a customer-by-product quantity matrix. Conversion analysis uses one row per web session with a forward-looking seven-day completed-order attribution.

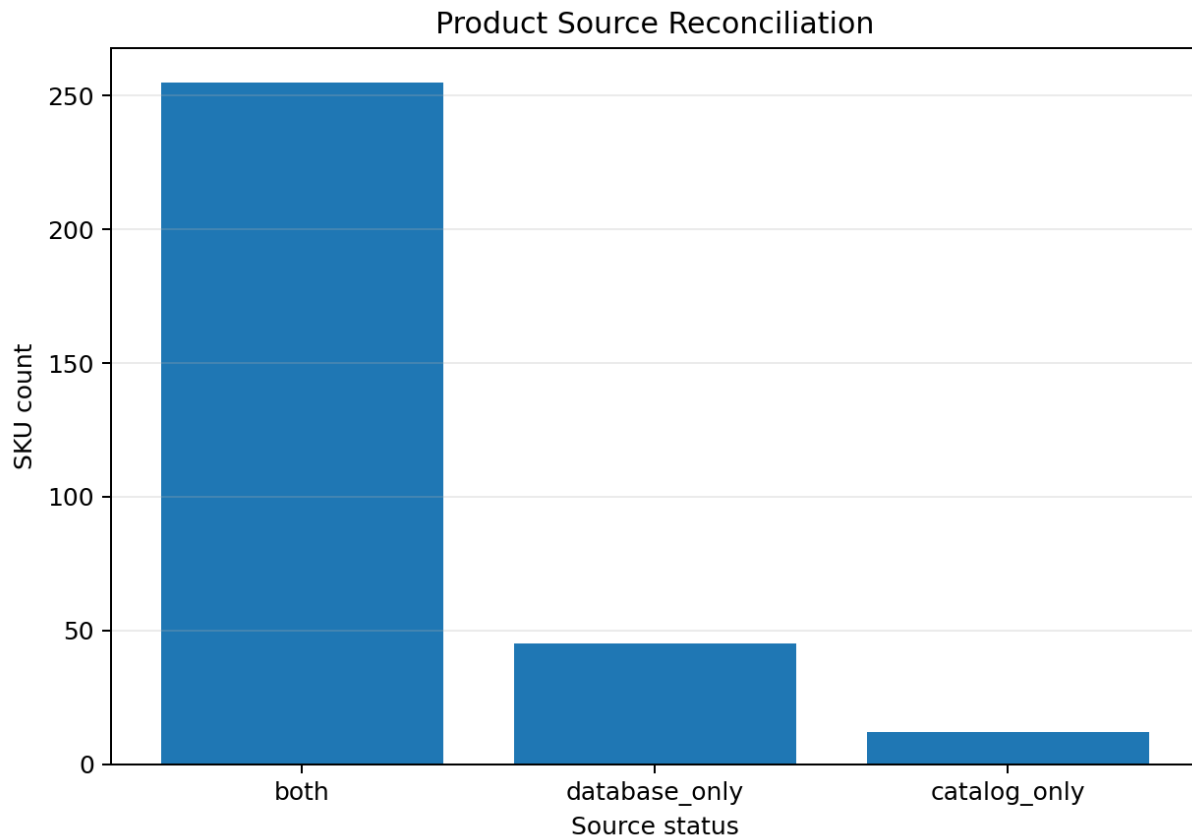


Figure 2. Product source overlap after SKU reconciliation.

2.3 Known Limitations in Raw Data

The database contains 186 duplicate business rows in order_items when the surrogate row identifier is ignored. Customer fields have missing city, age, and gender values. Reviews contain 41 ratings outside the valid 1-5 range and 838 missing review texts. The product table contains 3 extreme prices beyond the 1.5-IQR upper bound of \$1,160.36.

The catalog overlaps with 255 database SKUs, while 45 exist only in the database and 12 exist only in the supplier catalog. The legacy customer file begins with 1427 rows and is reduced to 1375 valid, deduplicated records.

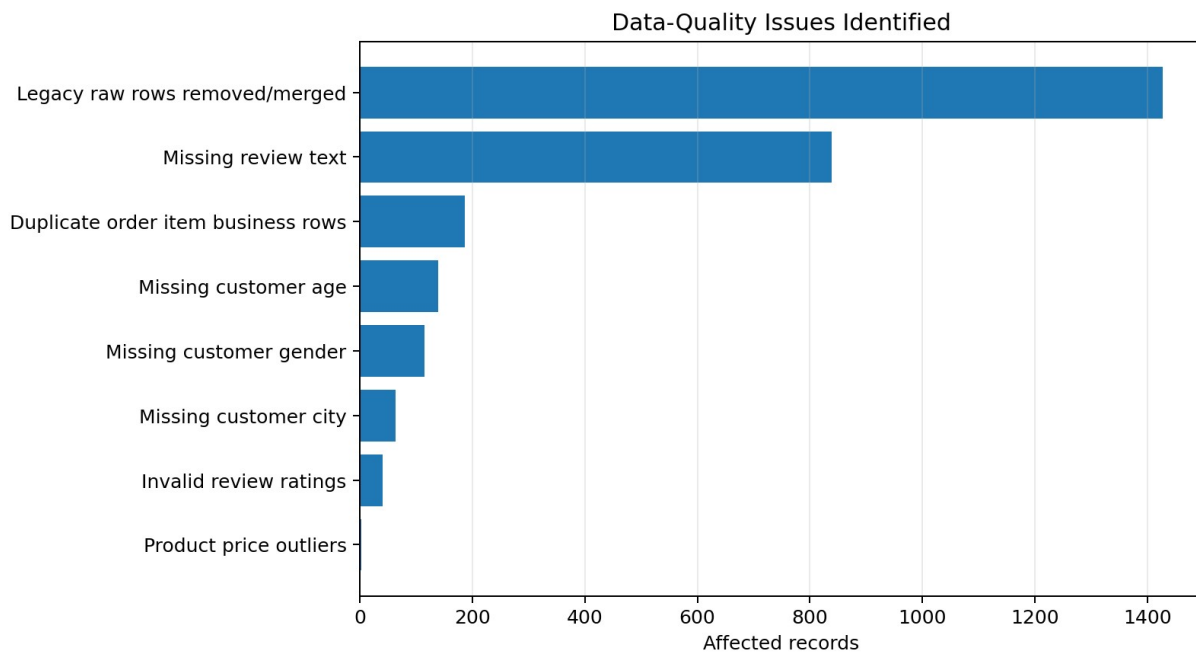


Figure 3. Number of records affected by each major data-quality issue.

3. SQL Extraction and Methodology

3.1 Query 1: Calculate recognized revenue, completed-order count, and average order value by product category, net of discounts.

Raw completed-order revenue is highest for Books & Media at \$2,288,541.26. This raw result is later shown to be distorted by extreme product prices, demonstrating why SQL extraction and cleaning must remain connected.

Category	Total Revenue	Order Count	Average Order Value
Books & Media	2,288,541.260	1649	1,387.840
Beauty & Health	1,781,007.780	1637	1,087.970
Apparel	967,057.000	1629	593.650
Sports & Outdoors	916,247.780	1579	580.270
Electronics	900,204.480	1615	557.400
Home & Kitchen	781,563.910	1660	470.820

```
WITH category_order_revenue AS (
    SELECT
        p.category,
        o.order_id,
        SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS order_revenue
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.order_id
    JOIN products AS p ON p.product_id = oi.product_id
    WHERE o.status = 'completed'
    GROUP BY p.category, o.order_id
)
SELECT
    category,
    ROUND(SUM(order_revenue), 2) AS total_revenue,
    COUNT(DISTINCT order_id) AS order_count,
    ROUND(AVG(order_revenue), 2) AS average_order_value
FROM category_order_revenue
GROUP BY category
ORDER BY total_revenue DESC;
```

3.2 Query 2: Identify the top 20 customers by lifetime spend, including city and signup date.

The highest raw lifetime-spend customer is Tara Holmes with \$145,169.05 across 3 completed orders. The list is an operational starting point, but cleaned RFM scoring provides a more balanced segmentation.

Customer Id	Customer Name	Email	City	Signup Date	Completed Orders	Lifetime Spend
611	Tara Holmes	arthurgreene@example.net	Sydney	2021-10-15	3	145,169.050
689	Misty Mcguire	tracynolan@example.net	Toronto	2020-08-25	2	107,213.910
1259	Matthew Perez	xbrooks@example.com	Los Angeles	2023-02-25	2	94,519.390
1217	Virginia Hernandez	kshelton@example.org	Paris	2022-07-07	4	77,530.810
528	Brent Hopkins	peteronmaria@example.com	Vancouver	2023-10-12	6	72,405.360
2445	Virginia Rose	nicoleduran@example.net	Bangkok	2023-11-26	3	72,021.090
1993	Chelsea Austin	leahbarker@example.org	Sydney	2020-04-27	1	70,254.390
1347	Evan Davies	vlewis@example.com	Paris	2020-05-28	2	70,236.440

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.email,
    c.city,
    c.signup_date,
    COUNT(DISTINCT o.order_id) AS completed_orders,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))), 2) AS lifetime_spend
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.customer_id
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.status = 'completed'
GROUP BY c.customer_id, c.name, c.email, c.city, c.signup_date
ORDER BY lifetime_spend DESC
LIMIT 20;
```

3.3 Query 3: Show the month-over-month revenue trend for the latest 24 months using LAG.

The query uses LAG to compare each month with the prior month. Month-over-month values are volatile, so a rolling average and regression are used later rather than interpreting a single month's movement as a stable trend.

Revenue Month	Revenue	Previous Month Revenue	Month Over Month Change Percent
2023-01-01	141,785.110		
2023-02-01	135,209.460	141,785.110	-4.640
2023-03-01	172,275.150	135,209.460	27.410
2023-04-01	106,562.900	172,275.150	-38.140
2023-05-01	312,271.280	106,562.900	193.040
2023-06-01	226,846.850	312,271.280	-27.360
2023-07-01	200,398.050	226,846.850	-11.660
2023-08-01	164,611.430	200,398.050	-17.860

```
WITH monthly_revenue AS (
    SELECT
        DATE(o.order_date, 'start of month') AS revenue_month,
        SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS revenue
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.order_id
    WHERE o.status = 'completed'
    GROUP BY DATE(o.order_date, 'start of month')
),
latest_month AS (
    SELECT MAX(revenue_month) AS maximum_month FROM monthly_revenue
),
last_24_months AS (
    SELECT mr.*
    FROM monthly_revenue AS mr
    CROSS JOIN latest_month AS lm
    WHERE mr.revenue_month >= DATE(lm.maximum_month, '-23 months')
)
SELECT
    revenue_month,
    ROUND(revenue, 2) AS revenue,
    ROUND(LAG(revenue) OVER (ORDER BY revenue_month), 2) AS previous_month_revenue,
    ROUND(
        (revenue - LAG(revenue) OVER (ORDER BY revenue_month))
        / NULLIF(LAG(revenue) OVER (ORDER BY revenue_month), 0) * 100,
        2
    ) AS month_over_month_change_percent
FROM last_24_months
ORDER BY revenue_month;
```

3.4 Query 4: Calculate each category's return rate as the share of line items with negative quantity.

Beauty & Health has the highest negative-quantity line rate at 3.16%. This rate counts return lines rather than returned units, which is transparent and consistent with the assignment definition.

Category	Total Order Items	Returned Order Items	Return Rate Percent
Beauty & Health	3354	106	3.160
Electronics	3401	106	3.120
Apparel	3372	102	3.020
Books & Media	3455	93	2.690
Home & Kitchen	3468	91	2.620
Sports & Outdoors	3312	85	2.570

```
WITH category_returns AS (
    SELECT
        p.category,
        COUNT(*) AS total_order_items,
        SUM(CASE WHEN oi.quantity < 0 THEN 1 ELSE 0 END) AS returned_order_items
    FROM order_items AS oi
    JOIN products AS p ON p.product_id = oi.product_id
    GROUP BY p.category
)
SELECT
    category,
    total_order_items,
    returned_order_items,
    ROUND(returned_order_items * 100.0 / NULLIF(total_order_items, 0), 2) AS return_rate_percent
FROM category_returns
ORDER BY return_rate_percent DESC;
```

3.5 Query 5: Find customers who placed completed orders in every one of the latest three calendar quarters.

43 customers purchased in each of the latest three calendar quarters. The query converts year and quarter into a continuous quarter index, then requires three distinct periods in HAVING.

Customer Id	Customer Name	Email	Quarters With Orders
73	Alicia Gilmore	wwoods@example.com	3
89	Samuel Rivas	ramirezshannon@example.com	3
121	Thomas Romero	michellehill@example.com	3
142	Richard Johnson	jessica14@example.com	3
152	Teresa Ramirez	jeff73@example.com	3
365	Tracey Wagner	fortiz@example.com	3
405	Cassidy Horne	tranrobin@example.net	3
471	Jeremy McIntyre	wfigueroa@example.net	3

```
WITH latest_quarter AS (
    SELECT
        CAST(STRFTIME('%Y', MAX(order_date)) AS INTEGER) AS latest_year,
        CAST((CAST(STRFTIME('%m', MAX(order_date)) AS INTEGER) - 1) / 3 + 1 AS INTEGER) AS latest_q
    FROM orders
),
quarter_orders AS (
    SELECT
        o.customer_id,
        CAST(STRFTIME('%Y', o.order_date) AS INTEGER) * 4
        + CAST((CAST(STRFTIME('%m', o.order_date) AS INTEGER) - 1) / 3 AS INTEGER) AS
quarter_index
    FROM orders AS o
    WHERE o.status = 'completed'
),
quarter_bounds AS (
    SELECT latest_year * 4 + (latest_q - 1) AS latest_index FROM latest_quarter
)
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.email,
    COUNT(DISTINCT qo.quarter_index) AS quarters_with_orders
FROM quarter_orders AS qo
JOIN quarter_bounds AS qb
JOIN customers AS c ON c.customer_id = qo.customer_id
WHERE qo.quarter_index BETWEEN qb.latest_index - 2 AND qb.latest_index
GROUP BY c.customer_id, c.name, c.email
HAVING COUNT(DISTINCT qo.quarter_index) = 3
ORDER BY c.customer_id;
```

3.6 Query 6: Return the 10 highest-rated products that have at least 15 valid reviews.

10 products meet the minimum 15-review threshold and appear in the top-ten ranking. Invalid ratings are excluded so that out-of-range values do not affect the average.

Product Id	Product Name	Category	Review Count	Average Rating
200	White Fitnes	Sports & Outdoors	15	4.600
19	Improve Smartphone	Electronics	15	4.533
191	Yeah Camping	Sports & Outdoors	15	4.533
283	Strategy Children	Books & Media	15	4.533
21	Right Camera	Electronics	16	4.500
66	Wide Cookware	Home & Kitchen	15	4.400
182	Answer Fitnes	Sports & Outdoors	15	4.400
122	Attorney Men	Apparel	17	4.353

```
SELECT
    p.product_id,
    p.name AS product_name,
    p.category,
    COUNT(r.review_id) AS review_count,
    ROUND(AVG(r.rating), 3) AS average_rating
FROM products AS p
JOIN reviews AS r ON r.product_id = p.product_id
WHERE r.rating BETWEEN 1 AND 5
GROUP BY p.product_id, p.name, p.category
HAVING COUNT(r.review_id) >= 15
ORDER BY average_rating DESC, review_count DESC
LIMIT 10;
```

3.7 Query 7: Compare engagement by device among customers who made at least one completed purchase.

Among purchasers, desktop sessions have the highest average duration at 6.25 minutes. Duration does not by itself prove conversion, so Phase 4 adds a seven-day order-attribution rule.

Device	Session Count	Average Session Minutes	Average Pages Viewed
desktop	3494	6.250	4.680
mobile	3458	6.200	4.650
tablet	3569	6.090	4.740

```
SELECT
    ws.device,
    COUNT(*) AS session_count,
    ROUND(AVG(ws.duration_minutes), 2) AS average_session_minutes,
    ROUND(AVG(ws.pages_viewed), 2) AS average_pages_viewed
FROM web_sessions AS ws
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = ws.customer_id
        AND o.status = 'completed'
)
GROUP BY ws.device
ORDER BY average_session_minutes DESC;
```


3.8 Query 8: Rank products by recognized revenue within each category.

DENSE_RANK produces a within-category product ranking without gaps when revenue ties occur. The full output is saved in data/processed/sql_query_08.csv rather than truncated in the report.

Category	Product Id	Product Name	Revenue	Revenue Rank
Apparel	139	About Men	46,240.500	1
Apparel	104	Major Men	45,950.880	2
Apparel	136	Watch Footwear	45,845.050	3
Apparel	131	Spring Kid	43,443.580	4
Apparel	107	Appear Women	40,099.070	5
Apparel	142	Card Footwear	39,954.720	6
Apparel	123	Anything Footwear	39,229.850	7
Apparel	116	Ahead Men	36,435.130	8

```
WITH product_revenue AS (
  SELECT
    p.category,
    p.product_id,
    p.name AS product_name,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS revenue
  FROM products AS p
  JOIN order_items AS oi ON oi.product_id = p.product_id
  JOIN orders AS o ON o.order_id = oi.order_id
  WHERE o.status = 'completed'
  GROUP BY p.category, p.product_id, p.name
)
SELECT
  category,
  product_id,
  product_name,
  ROUND(revenue, 2) AS revenue,
  DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS revenue_rank
FROM product_revenue
ORDER BY category, revenue_rank, product_id;
```

3.9 Query 9: Calculate payment-method share of completed orders within each customer country.

Payment shares are calculated within each country using a partitioned window denominator. This prevents large countries from dominating the percentage calculation.

Country	Payment Method	Order Count	Country Order Share Percent
Australia	gift_card	165	22.270
Australia	credit_card	162	21.860
Australia	debit_card	141	19.030
Australia	paypal	137	18.490
Australia	bank_transfer	136	18.350
Canada	credit_card	152	21.410
Canada	gift_card	149	20.990
Canada	paypal	146	20.560

```
WITH payment_counts AS (
    SELECT
        c.country,
        o.payment_method,
        COUNT(*) AS order_count
    FROM orders AS o
    JOIN customers AS c ON c.customer_id = o.customer_id
    WHERE o.status = 'completed'
    GROUP BY c.country, o.payment_method
)
SELECT
    country,
    payment_method,
    order_count,
    ROUND(order_count * 100.0 / SUM(order_count) OVER (PARTITION BY country), 2) AS
country_order_share_percent
FROM payment_counts
ORDER BY country, country_order_share_percent DESC;
```

3.10 Query 10: Measure category profit and effective margin after discounts, product cost, and negative-quantity returns.

The raw database identifies Books & Media as the highest effective-margin category at 79.85%. Phase 4 repeats the calculation with cleaned prices and duplicate removal.

Category	Net Revenue	Net Cost	Net Profit	Effective Margin Percent
Books & Media	2,288,541.260	461,053.850	1,827,487.410	79.850
Beauty & Health	1,781,007.780	493,857.040	1,287,150.740	72.270
Apparel	967,057.000	493,913.060	473,143.940	48.930
Electronics	900,204.480	428,163.670	472,040.810	52.440
Sports & Outdoors	916,247.780	456,371.420	459,876.360	50.190
Home & Kitchen	781,563.910	414,986.110	366,577.800	46.900

```
SELECT
    p.category,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))), 2) AS net_revenue,
    ROUND(SUM(oi.quantity * p.cost), 2) AS net_cost,
    ROUND(SUM(oi.quantity * (oi.unit_price * (1 - COALESCE(oi.discount, 0)) - p.cost)), 2) AS
net_profit,
    ROUND(
        SUM(oi.quantity * (oi.unit_price * (1 - COALESCE(oi.discount, 0)) - p.cost))
        / NULLIF(SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))), 0) * 100,
        2
    ) AS effective_margin_percent
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
WHERE o.status = 'completed'
GROUP BY p.category
ORDER BY net_profit DESC;
```

4. Data Cleaning and Integration

4.1 Legacy Date Standardization

The legacy signup column contains four patterns: DD-Mon-YYYY, Month DD, YYYY, MM/DD/YYYY, and ISO YYYY-MM-DD. A deterministic parser tests those formats in order. Slash dates are interpreted as month/day/year because values such as 01/25/2021 make day/month impossible. Unparseable values are retained as missing and can be audited through the raw-date column.

This approach is preferred over a generic parser because it makes ambiguity explicit and keeps the pipeline reproducible across pandas versions.

Name Clean	Email	Signup Date Raw	Signup Date	City Clean	Marketing Segment
Victoria Massey	oheath@example.com	01-Jan-2019	2019-01-01	Sydney	B
Vicki Hernandez	nicholas09@example.org	January 08, 2019	2019-01-08	Manchester	C
Kimberly Chan	gouldchristina@example.com	January 11, 2019	2019-01-11	Chicago	VIP
Timothy White	hilljamie@example.net	14-Jan-2019	2019-01-14	Los Angeles	B
Alexandra Hodge	robertsonjennifer@example.com	01/15/2019	2019-01-15	Berlin	VIP
Luke Miller	pparker@example.net	2019-01-20	2019-01-20	New York	C
Krystal Levine	april91@example.com	01/20/2019	2019-01-20	Bangkok	C
Adrian Lee	lori69@example.net	January 22, 2019	2019-01-22	Mumbai	A
David Small	brownalexa@example.com	2019-01-23	2019-01-23	Mumbai	C
Christopher Mason	michaelmccarthy@example.com	01/25/2019	2019-01-25	London	B

4.2 Duplicate and Near-Duplicate Customer Resolution

The legacy file begins with 1427 rows. Blank rows and the explicit test account are removed. Names, emails, and cities are trimmed and normalized to lowercase for matching. Email is the primary duplicate key. Records without an email use normalized name plus city. Within each city, names with at least 0.94 sequence similarity are treated as near duplicates only when an email is missing or agrees.

The result contains 1375 cleaned legacy customers. 1373 match the current customer database and 2 remain legacy-only. All unmatched records receive a generated LEG key so they can be preserved without pretending that they have a database customer_id.

Legacy Key	Name Clean	Email Normalized	City Clean	Marketing Segment	Matched Customer Id
LEG00001	Victoria Massey	oheath@example.com	Sydney	B	1558
LEG00002	Vicki Hernandez	nicholas09@example.org	Manchester	C	2338
LEG00003	Kimberly Chan	gouldchristina@example.com	Chicago	VIP	1555
LEG00004	Timothy White	hilljamie@example.net	Los Angeles	B	1052
LEG00005	Alexandra Hodge	robertsonjennifer@example.com	Berlin	VIP	2439
LEG00006	Luke Miller	pparker@example.net	New York	C	645
LEG00007	Krystal Levine	april91@example.com	Bangkok	C	519
LEG00008	Adrian Lee	lori69@example.net	Mumbai	A	1582
LEG00009	David Small	brownalexa@example.com	Mumbai	C	2452
LEG00010	Christopher Mason	michaelmccarthy@example.com	London	B	1460
LEG00011	Paul Phelps	john86@example.com	Chicago	VIP	2271
LEG00012	Debbie King	ucarr@example.org	Paris	A	2316

4.3 Missing-Value Policies

Missing city is imputed with the most common city in the same country. Missing age is imputed with the median for the same country and gender, then country median, then global median. Missing gender becomes Unknown. Each imputation has a companion flag so future models can distinguish observed from imputed values.

Missing review text is not a reason to discard a valid numeric rating. The row remains, text becomes an empty string, and review_text_missing is set. Ratings outside 1-5 are flagged and excluded from rating statistics rather than clipped or replaced with a fabricated value.

Issue	Before Count	After Count	Policy
Missing customer city	63	0	Impute country mode; retain city_missing flag
Missing customer age	139	0	Median within country/gender, then country/global fallback; retain age_missing flag
Missing customer gender	115	0	Use Unknown; retain gender_missing flag
Missing review text	838	0	Keep rating; blank text plus review_text_missing flag
Invalid review ratings	41	41	Flag and exclude from rating statistics; do not fabricate rating
Duplicate order item business rows	186	0	Keep first row by business fields; preserve negative quantities as returns
Product price outliers	3	0	IQR flag; replace analytical price with non-outlier subcategory median; retain raw price and flag
Legacy raw rows removed/merged	1427	1375	Remove blank/test rows; deduplicate by normalized email, then name+city, then high-similarity check

4.4 Product Price Outliers

The 1.5-IQR rule produces lower and upper bounds of \$-388.48 and \$1,160.36. Products 245, 250, 270 exceed the upper bound. Raw prices are retained in unit_price_raw. For analytical revenue, each outlier price is replaced with the median non-outlier price in the same subcategory.

A subcategory median is more defensible than a global cap because it preserves the normal price level of comparable products. The cleaned value is used in financial analysis and the raw value remains available for audit and source correction.

Product Id	Product Name	Category Clean	Unit Price Raw	Unit Price Clean	Cost Clean
245	Throughout Personal Care	Beauty & Health	11,506.500	390.340	104.230
250	Discover Haircare	Beauty & Health	8,284.000	279.955	112.790
270	Environmental Non-Fiction	Books & Media	34,872.000	274.430	252.130

4.5 Product Catalog Reconciliation

SKU is renamed to product_id and dept is renamed to category. Of all records, 255 SKUs appear in both sources, 45 are database-only, and 12 are supplier-only. The pipeline does not silently discard one-source records; source_status identifies each case.

For overlapping records, product name, category, and supplier cost agree strongly, while list prices differ modestly. Current stock is available only from the catalog and therefore stockout analysis is limited to catalog-covered products.

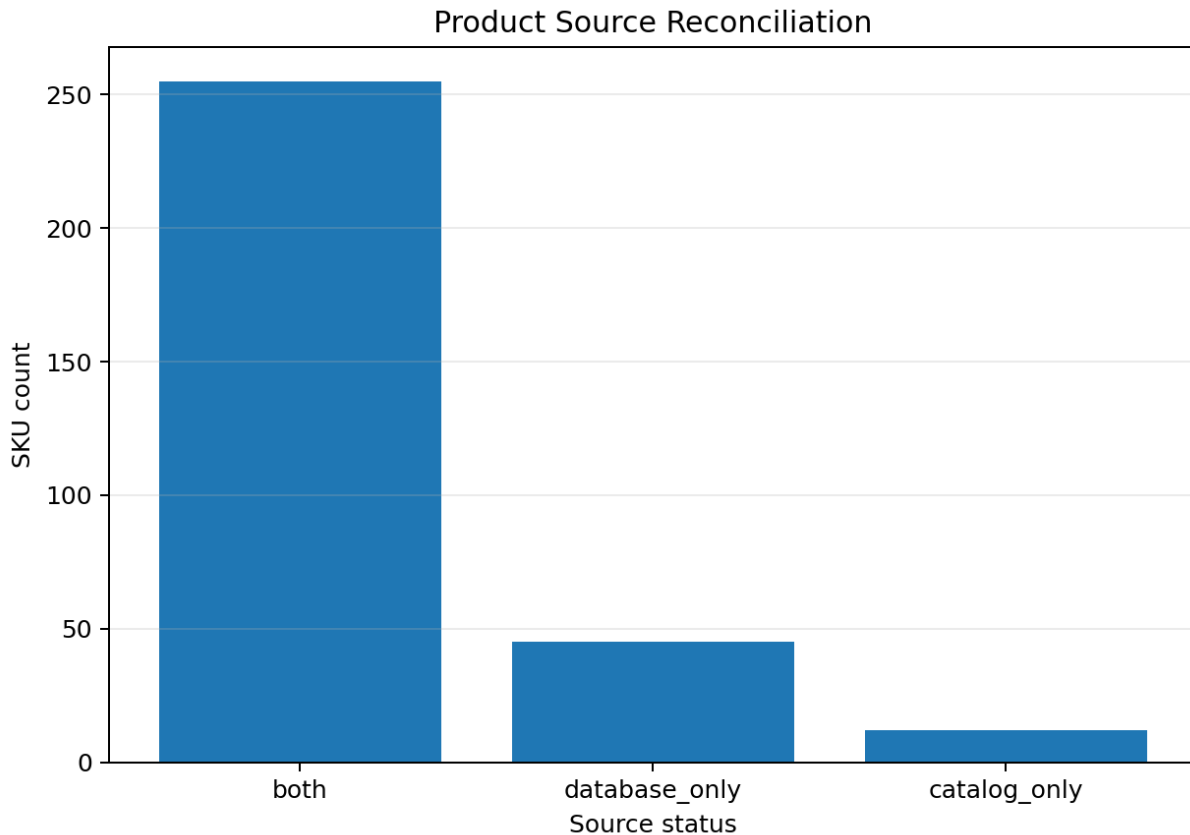


Figure 4. Product master reconciliation by source status.

Product Id	Product Name	Category Clean	Source Status	In Stock Units
4	Movement Headphone	Electronics	database_only	
6	Important Laptop	Electronics	database_only	
13	Support Headphone	Electronics	database_only	
16	Yes Smartphone	Electronics	database_only	
17	Light Laptop	Electronics	database_only	
27	Into Laptop	Electronics	database_only	
32	East Smartphone	Electronics	database_only	

4.6 Order-Item Duplicates and Returns

The surrogate `order_item_id` prevents exact full-row duplicates, but 186 rows duplicate the business fields `order_id`, `product_id`, `quantity`, `unit_price`, and `discount`. The pipeline keeps the first occurrence and removes subsequent duplicates.

Negative quantities remain negative. Net revenue equals quantity x adjusted unit price x (1 - discount), and net cost equals quantity x product cost. A return therefore reverses both revenue and cost. Completed orders are recognized; pending and cancelled orders are not. Returned order status is not automatically converted to a negative line because the assignment explicitly identifies quantity as the return indicator.

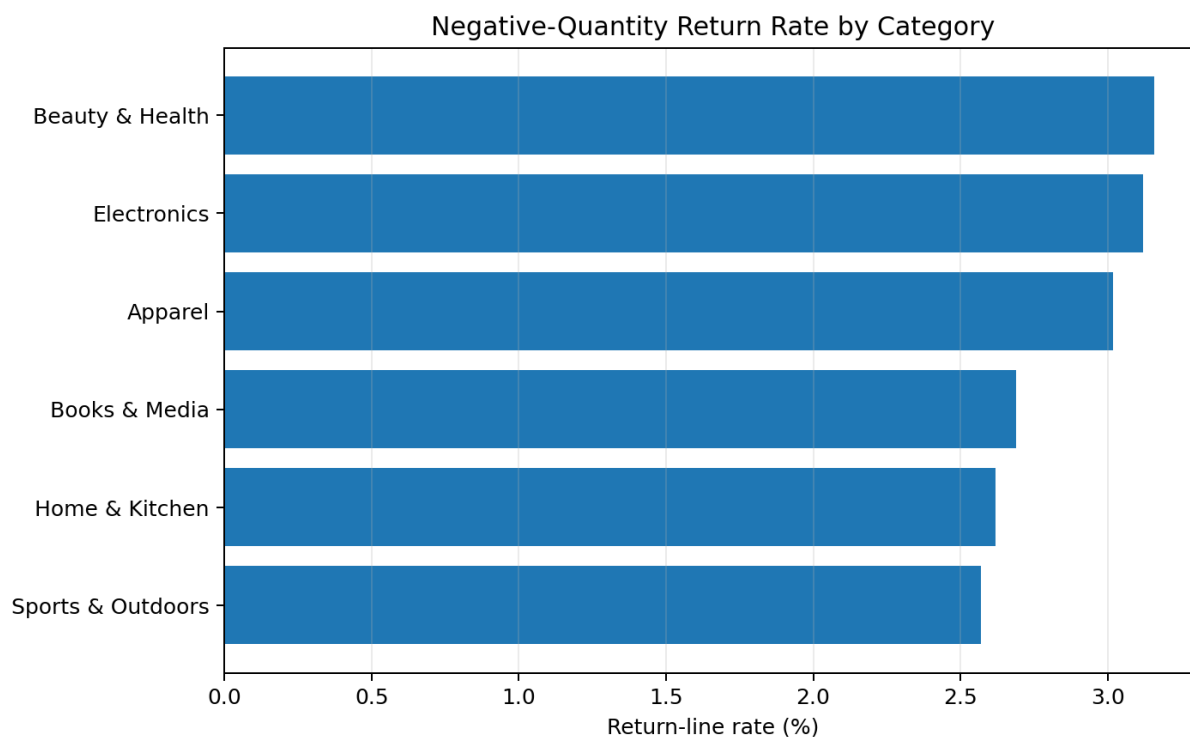


Figure 5. Return-line rate using negative quantities.

4.7 Reshaping and Time-Series Outputs

The pipeline produces a category-by-month revenue pivot table, satisfying the reshape requirement, and a weekly active-customer series using a datetime index and resampling. Both files are saved in `data/processed` and can be reused without reopening the database.

These derived tables are deliberately separated from raw inputs. The design makes it possible to rerun cleaning, inspect outputs, and use the processed files in downstream reporting or models.

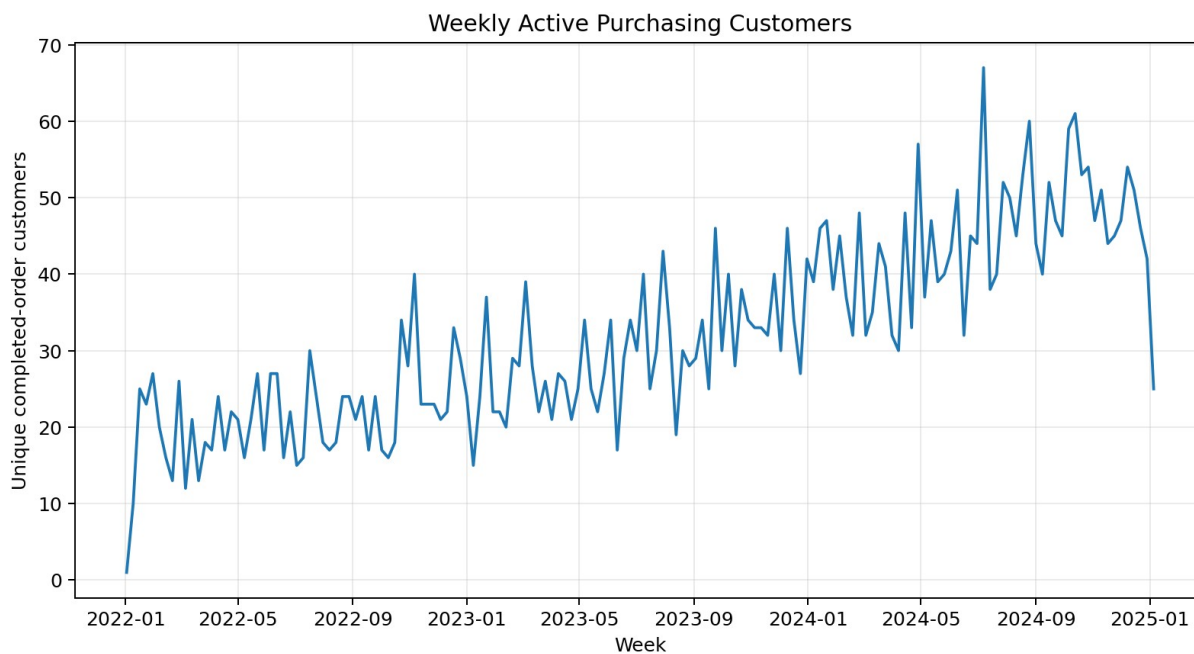


Figure 6. Weekly active customers from completed orders.

5. NumPy Analytical Methods

5.1 RFM Formula and Scoring

Recency is the number of days between the reference date and the customer's latest completed order. Frequency is the number of distinct completed orders. Monetary value is cleaned net revenue. The vectors R, F, and M are converted to quintiles with NumPy percentiles and searchsorted. Recency is reversed so that fewer days receives a higher score.

The combined score is $0.30R + 0.30F + 0.40M$. Monetary value receives the largest weight because leadership asked specifically about value. Segment boundaries are Champions ≥ 4.5 , Loyal ≥ 3.5 , Potential ≥ 2.5 , At Risk ≥ 1.5 , and Hibernating below 1.5.

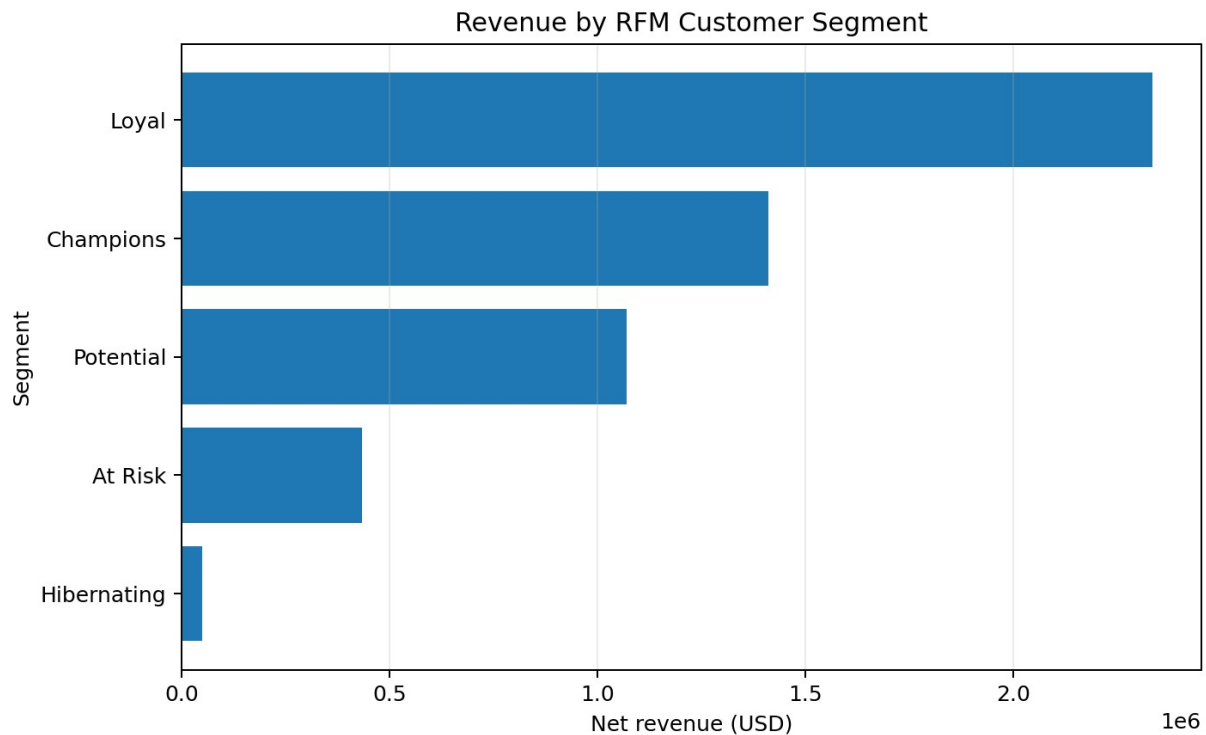


Figure 7. Segment revenue after NumPy scoring.

```
R_score = 6 - (searchsorted(percentile(R,[20,40,60,80]), R) + 1)
F_score = searchsorted(percentile(F,[20,40,60,80]), F) + 1
M_score = searchsorted(percentile(M,[20,40,60,80]), M) + 1
RFM = 0.30*R_score + 0.30*F_score + 0.40*M_score
```

5.2 RFM Validation and Results

Sanity check: in a trivial five-customer vector ordered from low to high, the percentile scoring must assign non-decreasing scores. For recency, reversing the result must assign the highest score to the smallest number of days. The implementation verifies these conditions in the notebook.

The model scores 2191 purchasing customers. Loyal contributes the most total monetary value. The result table preserves the raw R, F, and M values as well as each score, making the segment auditable.

Customer Id	Name Clean	Recency	Frequency	Monetary	R Score	F Score	M Score	Rfm Weighted Score	Segment
2454	Sabrina Lopez	68	5	6,993.724	5	5	5	5.000	Champions
15	Rebecca Henderson	69	4	4,439.519	5	5	5	5.000	Champions
889	Michael Wilson	10	5	4,398.862	5	5	5	5.000	Champions
888	Andrea Reed	2	4	4,269.385	5	5	5	5.000	Champions
861	Mr. Bryan McKinney	17	3	4,903.175	5	5	5	5.000	Champions
1676	Richard Lyons	46	5	6,060.890	5	5	5	5.000	Champions
2077	Christina Carrillo	8	4	7,154.943	5	5	5	5.000	Champions
1198	Tina Olson	31	4	3,914.952	5	5	5	5.000	Champions
1911	Alexandra Green	62	3	5,596.190	5	5	5	5.000	Champions
912	Charles Morales	51	3	4,509.028	5	5	5	5.000	Champions
921	Matthew Chan	17	7	6,270.350	5	5	5	5.000	Champions
920	Barbara Cole	32	3	5,510.366	5	5	5	5.000	Champions

5.3 Cosine Similarity Formula

A customer-by-product quantity matrix X is created from positive completed purchase quantities. Product similarity is calculated directly with NumPy dot products and vector norms. For products i and j , similarity is $(X_i \text{ dot } X_j) / (||X_i|| ||X_j||)$. No sklearn similarity shortcut is used.

A customer recommendation score is the customer purchase vector multiplied by the product-similarity matrix. Products already purchased are masked, and the three highest remaining scores are returned.

```
product_dot = X.T @ X
norms = np.linalg.norm(X, axis=0)
S = product_dot / np.outer(norms, norms)
recommendation_scores = customer_vector @ S
```

5.4 Recommendation Results and Sanity Check

Sanity check: a vector compared with itself has cosine similarity 1, and orthogonal vectors have similarity 0. The diagonal is therefore expected to equal 1 for products with purchase history. Five customers with sufficient purchase history are sampled, with segment diversity where possible.

The recommendations are analytical candidates rather than guaranteed preferences. They reflect co-purchase patterns and do not include price, availability, or personal constraints.

Customer Id	Customer Name	Segment	Recommendation Rank	Product Id	Product Name	Similarity Score
73	Alicia Gilmore	Champions	1	215	Want Supplement	0.854
73	Alicia Gilmore	Champions	2	218	Month Supplement	0.811
73	Alicia Gilmore	Champions	3	225	Discover Skincare	0.729
1595	Lisa Johnson	Loyal	1	127	Research Footwear	0.979
1595	Lisa Johnson	Loyal	2	218	Month Supplement	0.913
1595	Lisa Johnson	Loyal	3	53	Student Decor	0.899
2370	Jason Reyes	Potential	1	253	Happen Fiction	0.666
2370	Jason Reyes	Potential	2	15	Sea Headphone	0.547
2370	Jason Reyes	Potential	3	20	Arm Accessorie	0.473
31	Angela Morton	At Risk	1	115	Quickly Footwear	0.305
31	Angela Morton	At Risk	2	132	Set Men	0.284
31	Angela Morton	At Risk	3	149	Move Kid	0.284
831	Donald Todd	Hibernating	1	237	Against Skincare	0.216
831	Donald Todd	Hibernating	2	53	Student Decor	0.200
831	Donald Todd	Hibernating	3	236	Down Haircare	0.183

5.5 Normal-Equation Regression

The model predicts monthly cleaned revenue from a month index. With a design matrix containing an intercept and month number, $\beta = (X^T X)^{-1} X^T y$. The implementation uses the NumPy pseudoinverse for numerical stability while preserving the same normal-equation solution.

R-squared is calculated manually as $1 - \text{SSE}/\text{SST}$. A 95% prediction interval uses residual variance and the leverage of each future design row. The model is intentionally simple and serves as a transparent baseline rather than a production seasonal forecast.

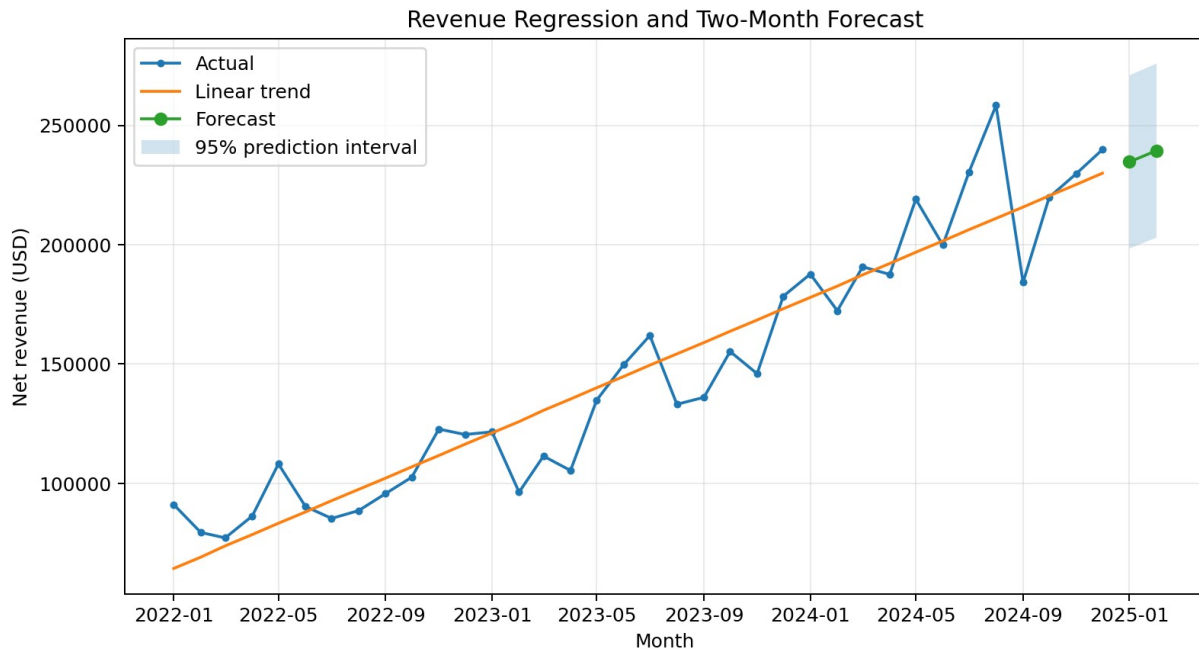


Figure 8. Normal-equation trend and two-month forecast.

```
beta = pinv(X.T @ X) @ (X.T @ y)
y_hat = X @ beta
R2 = 1 - sum((y-y_hat)^2) / sum((y-mean(y))^2)
```

5.6 Regression Results and Validation

The fitted monthly slope is \$4,731.68 per month and R-squared is 0.893. The forecasts are \$234,675.01 for 2025-01 and \$239,406.69 for 2025-02.

Sanity check: predicted values generated by $X @ \beta$ are compared with a NumPy polyfit line in the notebook for verification only. Because R-squared is low, seasonality and random variation remain important; the prediction intervals are more informative than point forecasts alone.

Month	Forecast Revenue	Lower 95	Upper 95
2025-01-01	234,675.011	198,395.818	270,954.204
2025-02-01	239,406.691	202,968.842	275,844.540

5.7 Monte Carlo Stockout Model

For every catalog-covered product, recent positive completed demand over the last 180 days is converted to an average daily rate. Ten thousand 30-day demand paths are simulated from a Poisson distribution. Stockout probability is the share of trials whose total demand exceeds current stock.

The recommended reorder point is mean simulated 30-day demand + 1.645 x simulated standard deviation, corresponding to an approximate 95% one-sided service target. The simulation also reports a binomial confidence interval for the probability estimate and a 95% range for demand.

```
daily_trials = rng.poisson(lambda_daily, size=(10000, 30))
thirty_day_demand = daily_trials.sum(axis=1)
P(stockout) = mean(thirty_day_demand > current_stock)
reorder_point = ceil(mean + 1.645*std)
```

5.8 Monte Carlo Results and Sanity Check

The highest simulated risk is Those Non-Fiction with a 69.26% probability of stockout within 30 days. Products with large stock relative to recent demand have probabilities close to zero.

Sanity check: a product with zero daily demand should simulate zero demand and therefore have zero stockout probability when stock is non-negative. Increasing current stock while keeping the same random demand paths cannot increase the estimated stockout probability.

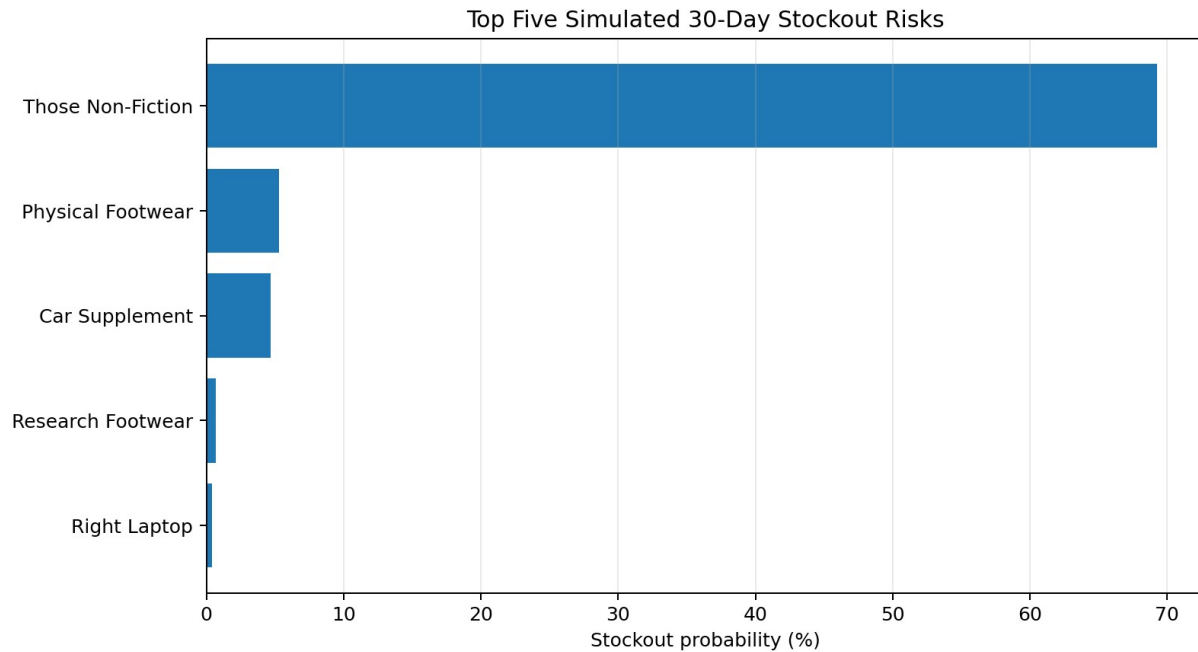


Figure 9. Highest 30-day stockout probabilities.

Product Id	Product Name	Current Stock	Mean 30 Day Demand	Stockout Probability	Recommended Reorder Point	Recommended Order Quantity
300	Those Non-Fiction	0	1.168	0.693	3	3
105	Physical Footwear	4	1.995	0.053	5	1
222	Car Supplement	3	1.321	0.047	4	1
127	Research Footwear	6	2.179	0.007	5	0
9	Right Laptop	5	1.486	0.004	4	0

6. Findings and Business Insights

6.1 Which Customer Segment Generates the Most Revenue?

Loyal is the largest revenue segment at \$2,334,550.52. The segment has strong recency, frequency, and monetary scores rather than relying on spend alone. Its demographic profile can be compared through average age, country, city, and gender fields in insight_01_rfm_segment_summary.csv.

Management should protect this group with service recovery, early access, and recognition rather than indiscriminate discounting. At Risk customers with historically high monetary values are the best win-back candidates.

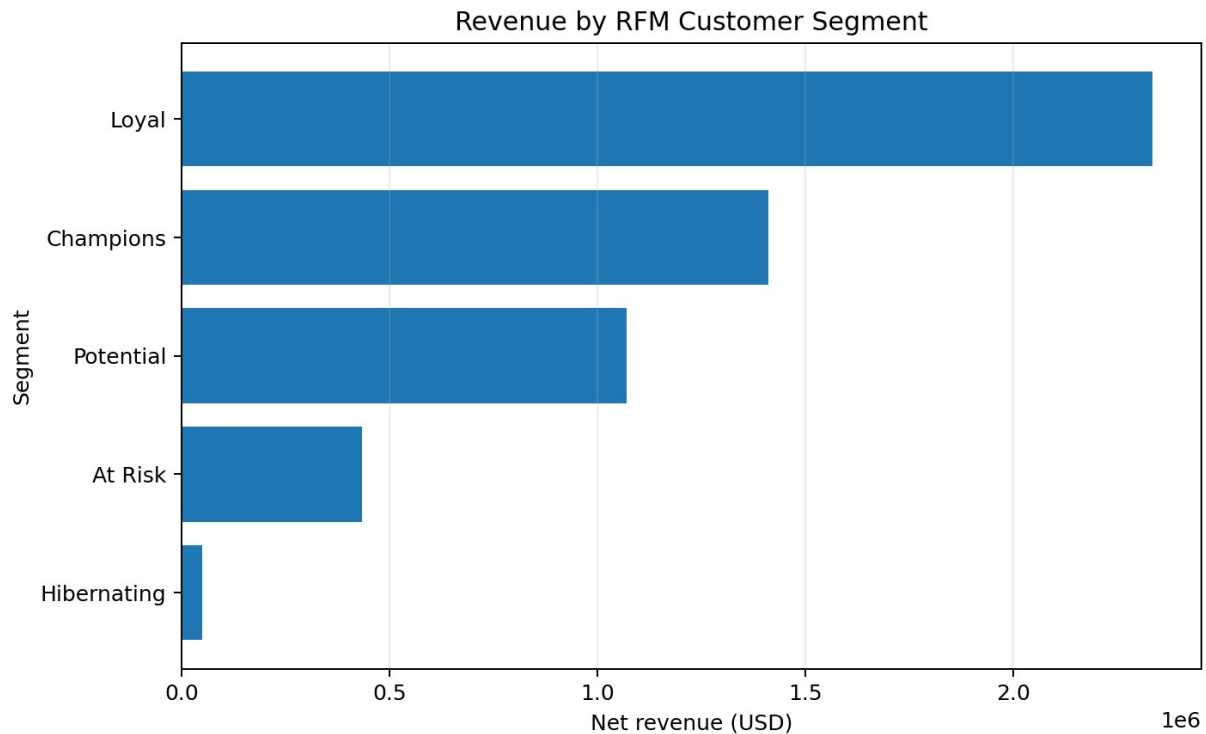


Figure 10. Revenue generated by each customer segment.

Segment	Customers	Revenue	Avg Revenue	Avg Age	Median Recency	Avg Frequency
Loyal	710	2,334,550.516	3,288.099	46.375	153.000	2.908
Champions	289	1,410,132.978	4,879.353	44.408	49.000	4.042
Potential	603	1,069,253.363	1,773.223	44.942	280.000	1.964
At Risk	451	433,500.437	961.198	44.120	404.000	1.211
Hibernating	138	49,564.097	359.160	45.152	768.500	1.000

6.2 Is There Seasonality in Overall Revenue?

Average revenue is highest in calendar month 12 and lowest in month 2. The monthly line is volatile, while the three-month rolling average reveals the underlying direction more clearly.

The available history covers three years, which is enough to suggest month-of-year differences but not enough to prove a stable long-run seasonal pattern. Promotional calendars and stock availability are not included and could explain part of the variation.

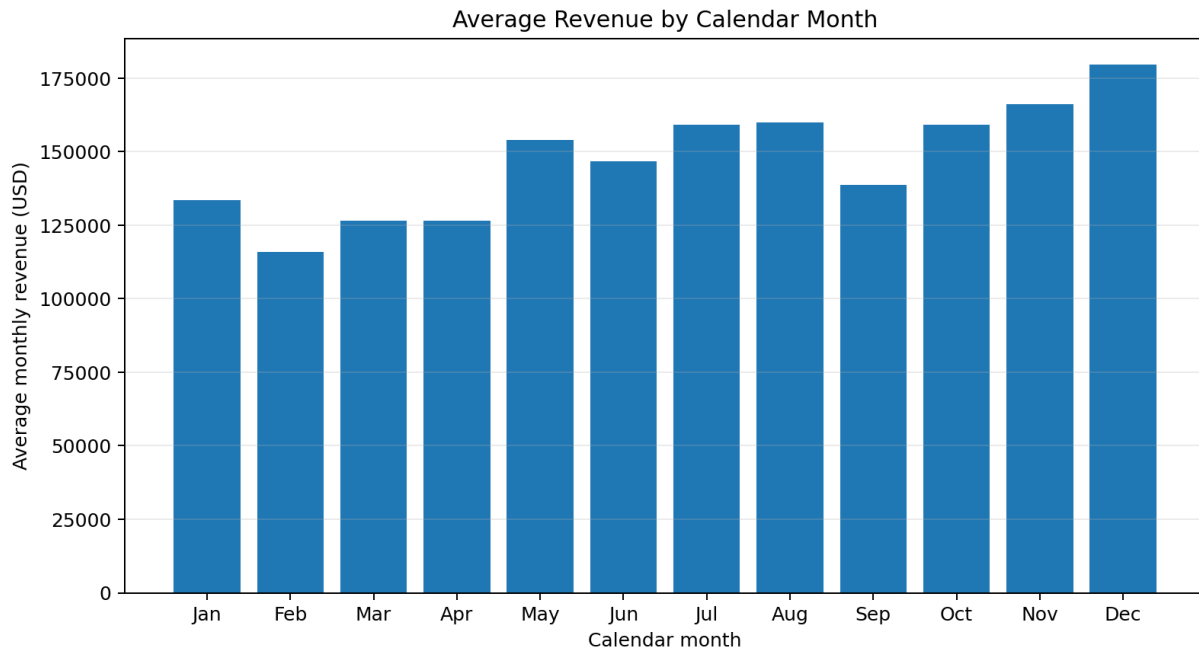


Figure 11. Average cleaned revenue by month of year.

6.3 Which Category Has the Highest Effective Margin?

Electronics has the highest cleaned effective margin at 52.40%. Effective margin is net profit divided by net revenue, where both revenue and cost are reversed by negative return quantities.

A high margin percentage should be considered alongside absolute profit and revenue. A small category can have an attractive margin but limited total contribution. The category-margin output therefore includes all three measures.

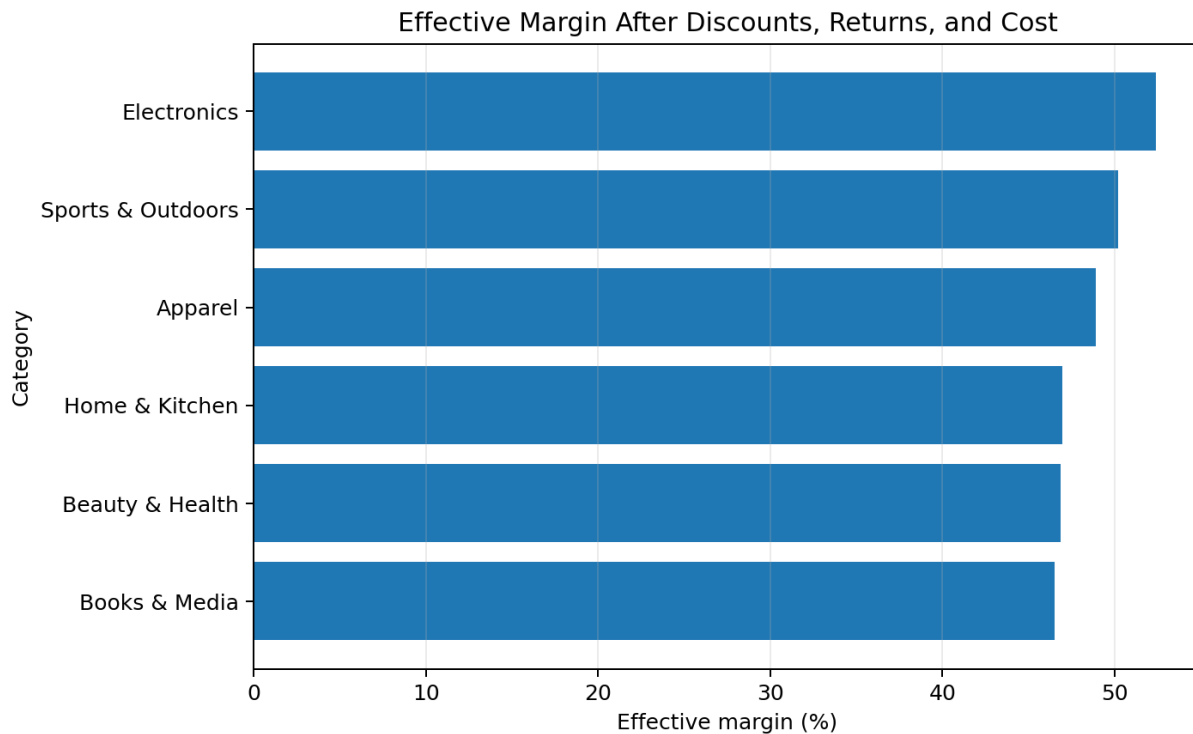


Figure 12. Effective margin by category.

Category	Net Revenue	Net Cost	Net Profit	Return Lines	Effective Margin Percent
Electronics	886,445.368	421,922.660	464,522.709	55	52.403
Sports & Outdoors	909,890.409	453,000.710	456,889.698	45	50.214
Apparel	960,533.889	490,706.150	469,827.738	68	48.913
Home & Kitchen	772,201.431	409,579.780	362,621.651	57	46.959
Beauty & Health	920,528.115	489,189.080	431,339.035	64	46.858
Books & Media	847,402.180	453,396.280	394,005.900	49	46.496

6.4 Do Higher Ratings Correlate With Repeat Purchase?

The product-level Pearson correlation between average valid rating and repeat-purchase rate is 0.076. This is weak, so ratings alone should not be used as a retention forecast.

Repeat rate is defined as the share of purchasing customers who bought the same product in at least two distinct completed orders. Product availability, replenishment cycles, and category purchase frequency can influence this measure independently of satisfaction.

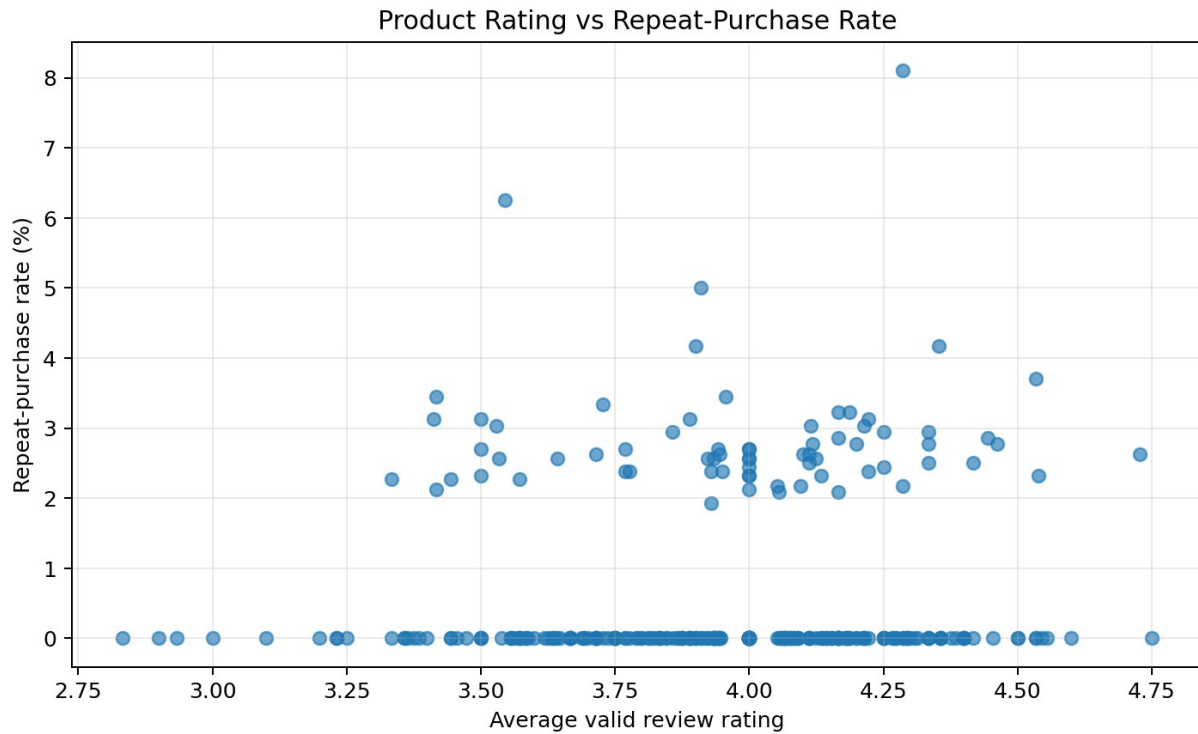


Figure 13. Product rating versus repeat-purchase rate.

6.5 Which Device/Country Combination Converts Best?

mobile / France has the highest eligible seven-day conversion rate at 2.85%. Only combinations with at least 30 sessions are ranked to reduce noise.

A session converts when the same customer places a completed order within seven days after the session. This customer-level attribution is more meaningful than simply restricting analysis to customers who have ever purchased, although it still cannot prove that the particular session caused the order.

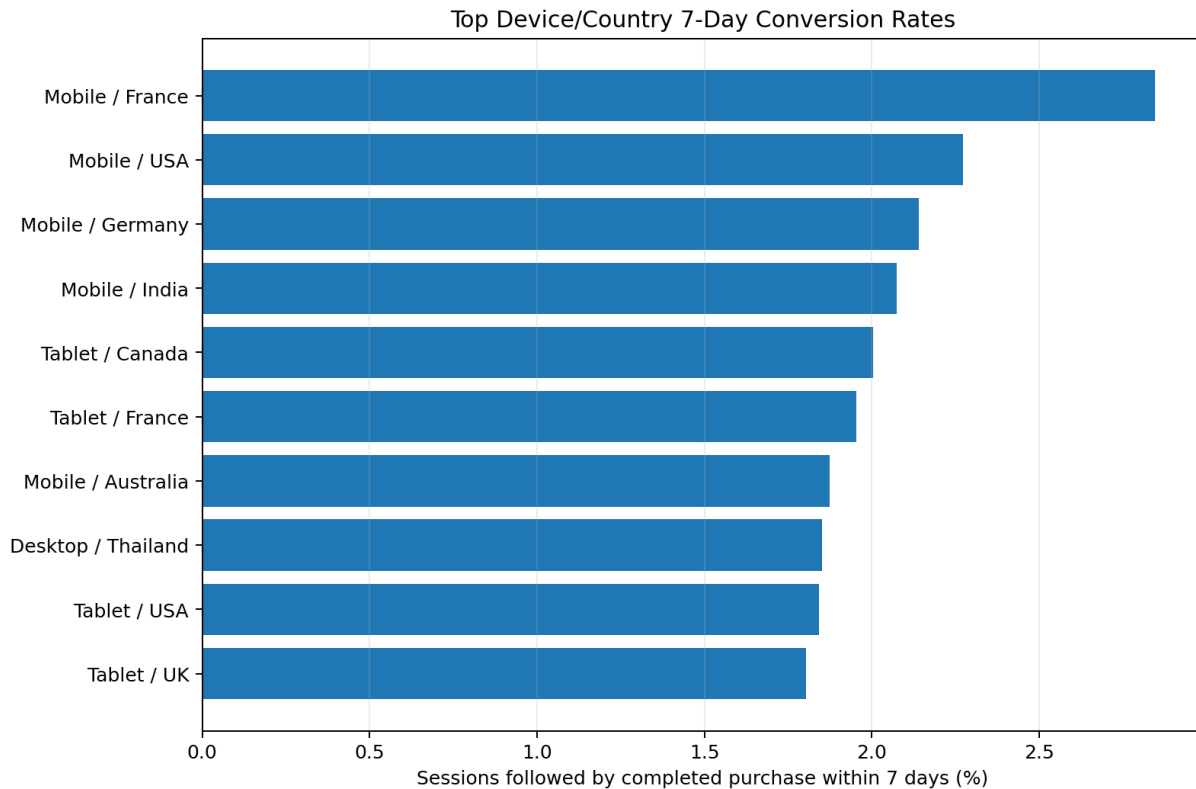


Figure 14. Highest seven-day session conversion combinations.

Device	Country	Sessions	Conversion Rate	Avg Duration	Avg Pages	Conversion Percent
mobile	France	281	0.028	6.374	4.523	2.847
mobile	USA	792	0.023	6.384	4.473	2.273
mobile	Germany	514	0.021	6.308	4.543	2.140
mobile	India	241	0.021	6.084	4.929	2.075
tablet	Canada	499	0.020	5.878	5.074	2.004
tablet	France	256	0.020	6.296	4.734	1.953
mobile	Australia	587	0.019	5.883	4.830	1.874
desktop	Thailand	270	0.019	6.617	5.037	1.852
tablet	USA	869	0.018	6.048	4.643	1.841
tablet	UK	499	0.018	6.372	4.723	1.804

6.6 What Is the Two-Month Revenue Projection?

The transparent linear baseline projects \$234,675.01 for 2025-01 and \$239,406.69 for 2025-02. The R-squared of 0.893 shows that month index explains only a limited share of revenue variation.

Leadership should use the point forecasts for planning scenarios only and should preserve the prediction interval. A seasonal model with promotion, category, price, and inventory features would be more suitable for operational forecasting.

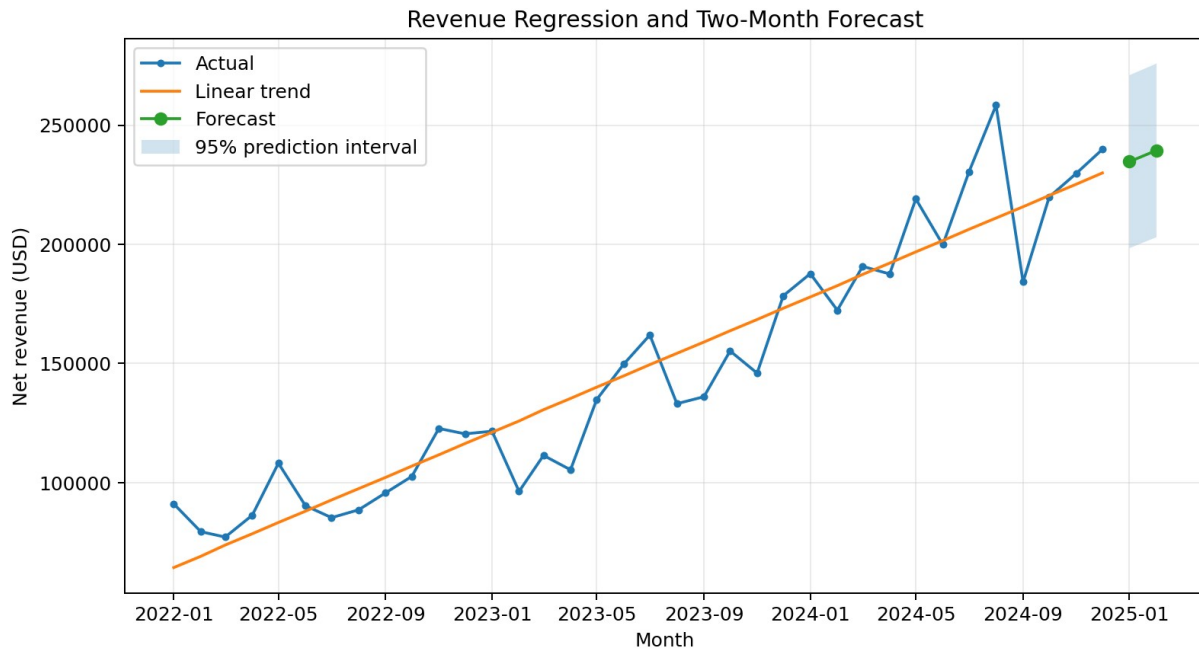


Figure 15. Revenue forecast with uncertainty range.

6.7 Which Products Have the Strongest Stockout Risk?

Those Non-Fiction is the highest-risk product because current stock is low relative to simulated demand. The top-five table shows current stock, mean 30-day demand, risk probability, and the recommended reorder point.

The recommended order quantity is reorder point minus current stock, floored at zero. This is a planning rule, not a purchase order: supplier lead time, minimum order quantity, shelf life, and service-level cost should be added before implementation.

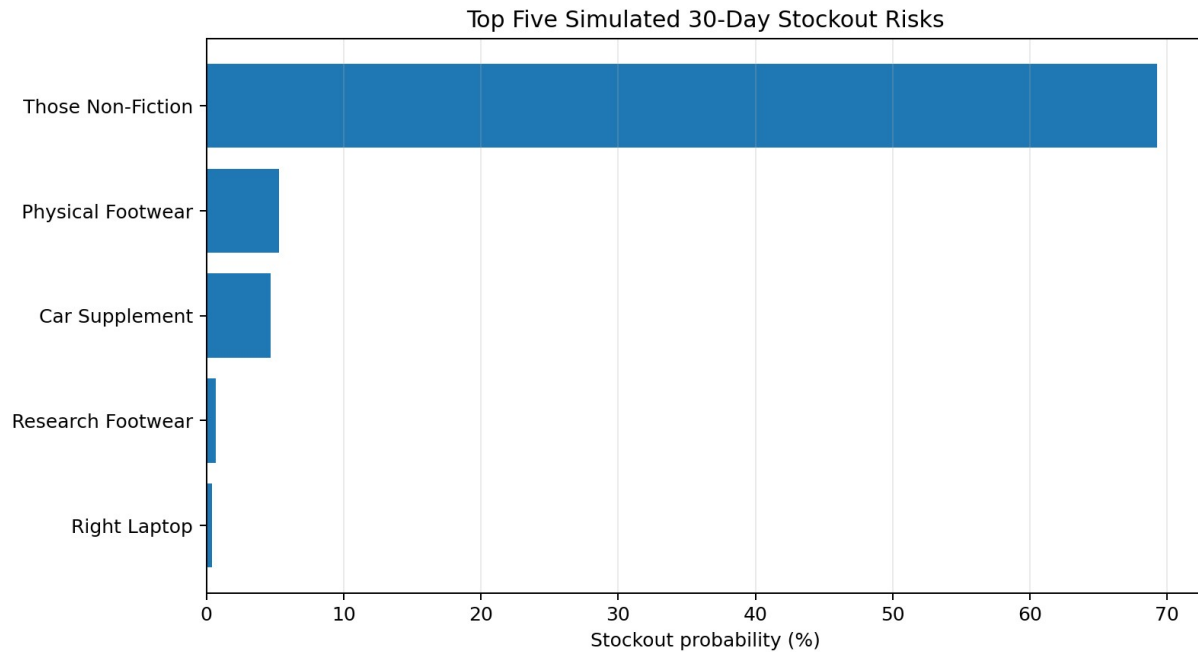


Figure 16. Top simulated stockout risks.

Product Id	Product Name	Category	Current Stock	Stockout Probability	Recommended Reorder Point	Recommended Order Quantity
300	Those Non-Fiction	Books & Media	0	0.693	3	3
105	Physical Footwear	Apparel	4	0.053	5	1
222	Car Supplement	Beauty & Health	3	0.047	4	1
127	Research Footwear	Apparel	6	0.007	5	0
9	Right Laptop	Electronics	5	0.004	4	0

6.8 Which Data-Quality Finding Changed a Conclusion?

Before price cleaning, Books & Media appears to be the highest-revenue category. After replacing three extreme prices with subcategory medians, Apparel becomes the leader. Total raw completed-order revenue changes from \$7,569,487.97 to \$5,297,001.39.

This is material because a raw-data decision could shift marketing budget, inventory attention, and category strategy toward a category whose apparent performance is driven by a few implausible product prices. The correction demonstrates why outlier policy must be documented and reversible.

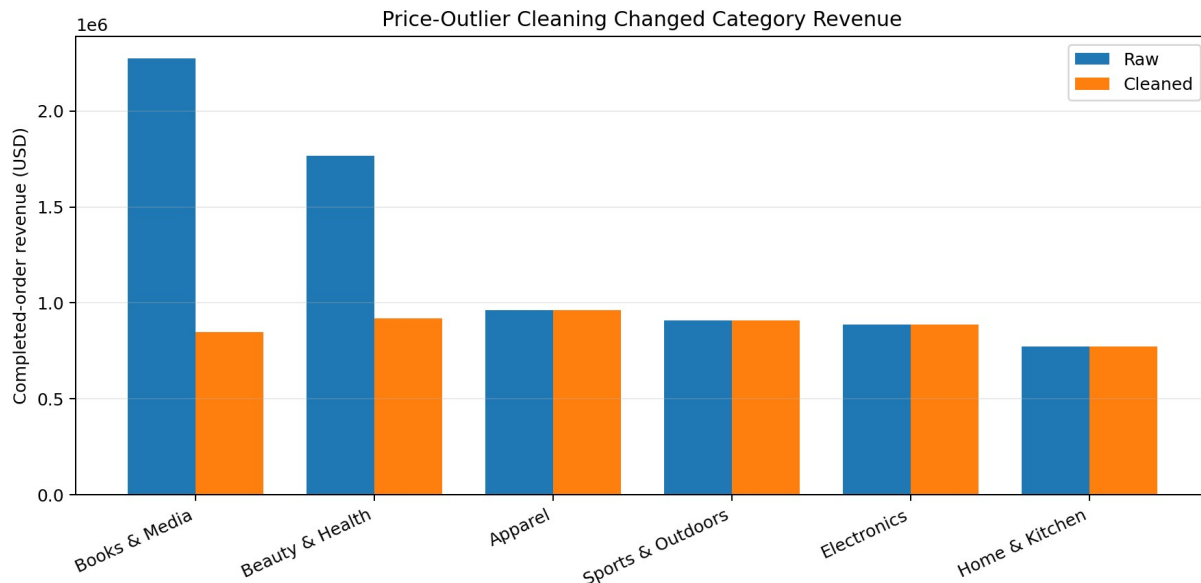


Figure 17. Revenue conclusion before and after outlier correction.

Category	Raw Revenue	Cleaned Revenue	Difference	Change Percent
Apparel	960,533.889	960,533.889	0.000	0.000
Beauty & Health	1,765,864.934	920,528.115	-845,336.818	-47.871
Books & Media	2,274,551.942	847,402.180	-1,427,149.762	-62.744
Electronics	886,445.368	886,445.368	0.000	0.000
Home & Kitchen	772,201.431	772,201.431	0.000	0.000
Sports & Outdoors	909,890.409	909,890.409	0.000	0.000

7. Limitations and Threats to Validity

7.1 Data and Measurement Limitations

The dataset is synthetic and does not include campaign exposure, customer acquisition cost, shipping cost, taxes, supplier lead time, inventory receipts, or product availability history. Profit therefore means merchandise margin after discount and product cost, not complete accounting profit. Web sessions have no session-level order identifier, so the seven-day attribution is an inference based on customer and time.

The price-outlier replacement is reasonable but cannot prove the true intended selling price. A source-system correction or invoice review would be preferable. Legacy records without stable emails can only be matched conservatively; overly aggressive fuzzy matching would risk merging different people.

- Returned order status is not automatically treated as a negative sale because quantity is the documented return indicator.
- RFM thresholds are relative to this customer population and will shift when data is refreshed.
- Catalog-only products have no sales history, and database-only products have no current stock for simulation.

7.2 Model Assumptions and Generalization

The linear regression assumes a straight trend, independent residuals, and roughly constant variance. Revenue shows month-to-month volatility and possible seasonality, so these assumptions are weak. The Monte Carlo model assumes Poisson demand with a constant recent rate and a fixed 30-day horizon. Real demand may be overdispersed, promotional, seasonal, or supply constrained.

Cosine recommendations reflect co-purchase similarity and favor products with broader history.

They do not measure causal preference and do not use content features. Rating-repeat correlation is product-level association and cannot establish that ratings cause repeat purchasing.

8. Recommendations and Next Steps

8.1 Prioritized Actions

The following actions are prioritized by decision impact and implementation feasibility.

- 1. Correct the three flagged product prices in the source system and add validation against cost, subcategory median, and maximum allowed markup. Re-run all financial outputs after correction.
- 2. Operationalize RFM segments monthly. Protect Champions, retain Loyal customers, nurture Potential customers, and create targeted win-back treatment for high-value At Risk customers.
- 3. Review the top-five stockout file immediately. Apply supplier lead time and minimum order quantity before creating purchase orders.
- 4. Build a controlled conversion dashboard using the same seven-day attribution and minimum-session threshold, then compare changes over time rather than relying on one snapshot.
- 5. Replace the baseline regression with a seasonal forecasting model once promotions, stock availability, and at least another year of history are available.

8.2 Data Governance and Pipeline Handoff

The processed outputs should be regenerated from raw files rather than edited manually. `queries.sql` is the source of truth for Phase 1. `src/urbancart_pipeline.py` contains reusable cleaning and numerical logic, `run_pipeline.py` executes the workflow, and `analysis.ipynb` is the documented driver for review.

A production handoff should add unit tests for row counts, primary-key uniqueness, allowable ratings, non-negative stock, price-to-cost ratios, and source overlap. Every run should save the data-quality summary and compare it with the prior run so new anomalies are visible.

Appendices

Appendix A - Data Dictionary

The following tables list the main analytical fields and their meaning.

Field	Meaning
customer_key	Stable DB or generated LEG identifier in integrated customer output
unit_price_raw	Original database product or order-item price
unit_price_clean / adjusted	Analytical price after outlier replacement policy
net_revenue	quantity x adjusted price x (1-discount)
net_profit	net revenue - quantity x product cost
rfm_weighted_score	$0.30R + 0.30F + 0.40M$
converted_7d	Completed order within seven days after a session
stockout_probability	Share of 10,000 simulations where 30-day demand exceeds stock

Appendix B - File Inventory

The repository contains raw data, all processed tables, 13 figures, SQL, a driver notebook, a reusable Python module, requirements, and the two PDF reports.

Path	Purpose
sql/queries.sql	All ten Phase 1 queries
notebooks/analysis.ipynb	Documented Phase 1-4 driver notebook
src/urbancart_pipeline.py	Reusable end-to-end pipeline
data/processed/*.csv	Cleaned and analytical outputs
figures/*.png	Business and data-quality charts
report/report.pdf	Standalone management report
report/executive_summary.pdf	One-page executive brief

Appendix C - Validation Checks

The notebook and pipeline include sanity checks that are intentionally simple enough to reason about independently.

- Database tables are checked for expected names before analysis.
- Cleaned order-item business keys are checked for zero duplicates.
- RFM score direction is checked on ordered trivial vectors.
- Cosine similarity diagonal is checked for products with non-zero purchase vectors.
- Regression predictions are compared with NumPy polyfit for verification only.
- Monte Carlo zero-demand and monotonic-stock conditions are checked.
- All processed files are written by the pipeline, not manually edited.

Appendix D - Individual Contribution Statement Template

Replace the placeholders below with the actual team member names before submission. Keep the final README contribution statement between 150 and 300 words as required.

Member 1: database inspection, SQL queries, and validation. Member 2: customer and product cleaning, source reconciliation, and documentation. Member 3: NumPy models, charts, and interpretation. Member 4: report assembly, GitHub organization, testing, and final quality review. Where fewer members participated, combine responsibilities and state the actual work completed by each person.